

要学习机器学习，需要掌握一些基本功能

- 关于数据的实用技能：包括存储、操作和预处理数据
- 矩阵运算等线性代数的基本原理及其实现上
- 微积分知识，并需要知道自动积分
- 概率语言

数据操作

1. 获取数据
2. 将数据读入计算机后对其进行处理

张量的定义：张量（Tensor）是一种多维数组，用于表示数据的结构和操作。张量的维度（Rank）是指张量的轴数，每个轴的长度（Size）是指该轴上的元素数量。张量的元素（Element）是指张量中每个位置上的数值。

入门

```
In [95]: import torch
```

```
x=torch.arange(12)
x
```

```
Out[95]: tensor([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```

```
In [96]: # 获取张量的形状
x.shape
```

```
Out[96]: torch.Size([12])
```

```
In [97]: # 获取张量的大小
x.numel()
```

```
Out[97]: 12
```

```
In [98]: # 改变张量的形状
X=x.reshape(4, 3)
print("X的形状为: ", X.shape)
X
```

X的形状为: torch.Size([4, 3])

```
Out[98]: tensor([[ 0,  1,  2],
                  [ 3,  4,  5],
                  [ 6,  7,  8],
                  [ 9, 10, 11]])
```

```
In [99]: # 形状的自动计算
X1=x.reshape(-1, 3)
print("x1的形状为: ", X1.shape)
X1
```

X1的形状为: torch.Size([4, 3])

```
Out[99]: tensor([[ 0,  1,  2],
                  [ 3,  4,  5],
                  [ 6,  7,  8],
                  [ 9, 10, 11]])
```

几种特殊的初始化张量的方法

```
In [100... # 全零张量
torch.zeros((2,3,4))
```

```
Out[100... tensor([[[0., 0., 0., 0.],
                  [0., 0., 0., 0.],
                  [0., 0., 0., 0.]],

                  [[0., 0., 0., 0.],
                  [0., 0., 0., 0.],
                  [0., 0., 0., 0.]])
```

```
In [101... # 全一张量
torch.ones((2,3,4))
```

```
Out[101... tensor([[[1., 1., 1., 1.],
                  [1., 1., 1., 1.],
                  [1., 1., 1., 1.]],

                  [[1., 1., 1., 1.],
                  [1., 1., 1., 1.],
                  [1., 1., 1., 1.]])
```

```
In [102... # 标准正态分布随机数的张量
torch.randn((2,3,4))
```

```
Out[102... tensor([[[ 0.0438,  1.4820, -0.3509, -1.0286],
                  [ 0.6716, -0.4750, -0.1932,  0.8915],
                  [ 0.7689,  0.1750, -1.0854, -2.7684]],

                  [[-0.7178, -0.0982, -0.0084, -0.5316],
                  [ 0.4248, -0.8635, -1.9853,  2.8358],
                  [-0.1737,  1.2268, -0.0378,  0.3918]]])
```

```
In [103... # 通过Python的列表生成张量

torch.tensor([[1,2,3],[4,5,6]])
```

```
Out[103... tensor([[1, 2, 3],
                  [4, 5, 6]])
```

运算符

按元素操作的运算符

+ 加法
- 减法
* 乘法
/ 除法
** 幂运算
cat 拼接
== 等于
!= 不等于
sum 求和

```
In [104... x=torch.tensor([[1,2,3],[4,5,6]])  
y=torch.tensor([[7,8,9],[10,11,12]])  
  
x+y
```

```
Out[104... tensor([[ 8, 10, 12],  
          [14, 16, 18]])
```

```
In [105... y-x
```

```
Out[105... tensor([[6, 6, 6],  
          [6, 6, 6]])
```

```
In [106... x*y
```

```
Out[106... tensor([[ 7, 16, 27],  
          [40, 55, 72]])
```

```
In [107... y/x
```

```
Out[107... tensor([[7.0000, 4.0000, 3.0000],  
          [2.5000, 2.2000, 2.0000]])
```

```
In [108... y**x
```

```
Out[108... tensor([[ 7, 64, 729],  
          [10000, 161051, 2985984]])
```

```
In [109... # 张量连接  
# 按照维度一连接  
torch.cat((x,y),dim=0)
```

```
Out[109... tensor([[ 1, 2, 3],  
          [ 4, 5, 6],  
          [ 7, 8, 9],  
          [10, 11, 12]])
```

```
In [110... # 按照维度二连接  
torch.cat((x,y),dim=1)
```

```
Out[110...] tensor([[ 1,  2,  3,  7,  8,  9],
          [ 4,  5,  6, 10, 11, 12]])
```

```
In [111...] # 逻辑运算符
x==y
```

```
Out[111...] tensor([[False, False, False],
          [False, False, False]])
```

```
In [112...] x!=y
```

```
Out[112...] tensor([[True, True, True],
          [True, True, True]])
```

```
In [113...] # 所有元素求和
x.sum()
```

```
Out[113...] tensor(21)
```

广播机制

(1) 通过适当复制元素来扩展一个或两个数组,以便在转换之后,两个张量具有相同的形状

(2) 对生成的数组执行暗元素操作

```
In [114...] a=torch.arange(3).reshape((3,1))
b=torch.arange(2).reshape((1,2))

print('a:',a)
print('b:',b)
print('a+b:',a+b)
```

```
a: tensor([[0],
          [1],
          [2]])
b: tensor([[0, 1]])
a+b: tensor([[0, 1],
          [1, 2],
          [2, 3]])
```

索引和切片

```
In [115...] X=torch.arange(12).reshape((3,4))
X
```

```
Out[115...] tensor([[ 0,  1,  2,  3],
          [ 4,  5,  6,  7],
          [ 8,  9, 10, 11]])
```

```
In [116...] X[-1]
```

```
Out[116...] tensor([ 8,  9, 10, 11])
```

```
In [117... X[1:3]

Out[117... tensor([[ 4,  5,  6,  7],
          [ 8,  9, 10, 11]])
```

```
In [118... X[1,2]=9
X

Out[118... tensor([[ 0,  1,  2,  3],
          [ 4,  5,  9,  7],
          [ 8,  9, 10, 11]])
```

```
In [119... X[0:2,:3]

Out[119... tensor([[0, 1, 2],
          [4, 5, 9]])
```

转换为其他Python对象

```
In [120... A=X.numpy()
B=torch.tensor(A)
print(type(A))
print(type(B))

<class 'numpy.ndarray'>
<class 'torch.Tensor'>
```

```
In [121... a=torch.tensor([3.5])
a
```

```
Out[121... tensor([3.5000])
```

```
In [122... print(a.item())
type(a.item())
```

3.5

```
Out[122... float
```

```
In [123... float(a)
```

```
Out[123... 3.5
```

数据预处理

读取数据集

```
In [124... import os

os.makedirs(os.path.join('.', 'data'), exist_ok=True)
data_file = os.path.join('.', 'data', 'house_tiny.csv')
with open(data_file, 'w') as f:
```

```
f.write('NumRooms,Alley,Price\n') # 列名
f.write('NA,Pave,127500\n') # 每行表示一个数据样本
f.write('2,NA,106000\n')
f.write('4,NA,178100\n')
f.write('NA,NA,140000\n')
```

```
In [125... # 如果没有安装pandas, 只需取消对以下行的注释来安装pandas
# !pip install pandas
import pandas as pd

data = pd.read_csv(data_file)
print(data)
```

	NumRooms	Alley	Price
0	NaN	Pave	127500
1	2.0	NaN	106000
2	4.0	NaN	178100
3	NaN	NaN	140000

处理缺失值

NAN代表缺失值

处理缺失的数据的典型方法包括插值法和删除法

- 插值法: 用缺失值的邻近值填充缺失值
- 删除法: 删除缺失值

```
In [126... inputs,outputs=data.iloc[:,0:2],data.iloc[:,2]
inputs=inputs.fillna(inputs.mean(numeric_only=True,skipna=True))
inputs
```

```
Out[126... NumRooms Alley
```

0	3.0	Pave
1	2.0	NaN
2	4.0	NaN
3	3.0	NaN

```
In [127... inputs=pd.get_dummies(inputs,dummy_na=True)
inputs
```

```
Out[127... NumRooms Alley_Pave Alley_nan
```

0	3.0	True	False
1	2.0	False	True
2	4.0	False	True
3	3.0	False	True

转换为张量格式

```
In [128... X, y = torch.tensor(inputs.to_numpy(dtype="float32")), torch.tensor(outputs.values)
X, y
```

```
Out[128... (tensor([[3., 1., 0.],
          [2., 0., 1.],
          [4., 0., 1.],
          [3., 0., 1.]]),
          tensor([127500, 106000, 178100, 140000]))
```

线性代数

标量

仅包含一个数值被称为标量

```
In [129... x=torch.tensor(3.0)
y=torch.tensor(2.0)
print(x*y)
print(x+y)
print(x-y)
print(x/y)
print(x**y)
```

```
tensor(6.)
tensor(5.)
tensor(1.)
tensor(1.5000)
tensor(9.)
```

向量

向量可以被视为标量值组成的列表

```
In [130... x=torch.arange(4)
x
```

```
Out[130... tensor([0, 1, 2, 3])
```

长度\维度\形状

```
In [131... len(x)
```

```
Out[131... 4
```

```
In [132... x.shape
```

```
Out[132... torch.Size([4])
```

矩阵

```
In [133... A=torch.arange(20).reshape(4,5)
A
```

```
Out[133... tensor([[ 0,  1,  2,  3,  4],
          [ 5,  6,  7,  8,  9],
          [10, 11, 12, 13, 14],
          [15, 16, 17, 18, 19]])
```

```
In [134... A.T
```

```
Out[134... tensor([[ 0,  5, 10, 15],
          [ 1,  6, 11, 16],
          [ 2,  7, 12, 17],
          [ 3,  8, 13, 18],
          [ 4,  9, 14, 19]])
```

```
In [135... B=torch.tensor([[1,2,3],[2,0,4],[3,4,5]])
B
```

```
Out[135... tensor([[1, 2, 3],
          [2, 0, 4],
          [3, 4, 5]])
```

张量算法的基本性质

```
In [136... A=torch.arange(20,dtype=torch.float32).reshape(4,5)
B=A.clone()
print('A:',A)
print('A+B:',A+B)
```

```
A: tensor([[ 0.,  1.,  2.,  3.,  4.],
           [ 5.,  6.,  7.,  8.,  9.],
           [10., 11., 12., 13., 14.],
           [15., 16., 17., 18., 19.]])
A+B: tensor([[ 0.,  2.,  4.,  6.,  8.],
            [10., 12., 14., 16., 18.],
            [20., 22., 24., 26., 28.],
            [30., 32., 34., 36., 38.]])
```

```
In [137... # 矩阵乘法
A@B.T
```

```
Out[137... tensor([[ 30.,  80., 130., 180.],
          [ 80., 255., 430., 605.],
          [130., 430., 730., 1030.],
          [180., 605., 1030., 1455.]])
```

```
In [138... # 矩阵数乘
2*A
```

```
Out[138...] tensor([[ 0.,  2.,  4.,  6.,  8.],
          [10., 12., 14., 16., 18.],
          [20., 22., 24., 26., 28.],
          [30., 32., 34., 36., 38.]])
```

```
In [139...] A*B
```

```
Out[139...] tensor([[ 0.,  1.,  4.,  9., 16.],
          [25., 36., 49., 64., 81.],
          [100., 121., 144., 169., 196.],
          [225., 256., 289., 324., 361.]])
```

降维

```
In [140...] x = torch.arange(4, dtype=torch.float32)
x, x.sum()
```

```
Out[140...] (tensor([0., 1., 2., 3.]), tensor(6.))
```

```
In [141...] A.shape
```

```
Out[141...] torch.Size([4, 5])
```

```
In [142...] A.sum()
```

```
Out[142...] tensor(190.)
```

```
In [143...] B=torch.arange(20,dtype=torch.float32).reshape(4,5)
B
```

```
Out[143...] tensor([[ 0.,  1.,  2.,  3.,  4.],
          [ 5.,  6.,  7.,  8.,  9.],
          [10., 11., 12., 13., 14.],
          [15., 16., 17., 18., 19.]])
```

```
In [144...] B.sum(axis=0)
```

```
Out[144...] tensor([30., 34., 38., 42., 46.])
```

```
In [145...] B.mean(axis=1)
```

```
Out[145...] tensor([ 2.,  7., 12., 17.])
```

非降维求和

```
In [146...] sum_B=B.sum(axis=1,keepdims=True)
sum_B
```

```
Out[146...] tensor([[10.],
          [35.],
          [60.],
          [85.]])
```

点积

```
In [147... y=torch.ones(4,dtype=torch.float32)
x=torch.arange(4,dtype=torch.float32)
print('x:',x)
print('y:',y)
```

```
x: tensor([0., 1., 2., 3.])
y: tensor([1., 1., 1., 1.])
```

```
In [148... torch.dot(x,y)
```

```
Out[148... tensor(6.)
```

矩阵乘法

```
In [149... A=torch.arange(12).reshape(3,4)
B=torch.arange(12).reshape(4,3)

print('A:',A)
print('B:',B)
```

```
A: tensor([[ 0,  1,  2,  3],
          [ 4,  5,  6,  7],
          [ 8,  9, 10, 11]])
B: tensor([[ 0,  1,  2],
          [ 3,  4,  5],
          [ 6,  7,  8],
          [ 9, 10, 11]])
```

```
In [150... torch.matmul(A,B)
```

```
Out[150... tensor([[ 42,  48,  54],
          [114, 136, 158],
          [186, 224, 262]])
```

```
In [151... A@B
```

```
Out[151... tensor([[ 42,  48,  54],
          [114, 136, 158],
          [186, 224, 262]])
```

范数

范数（norm）是衡量一个向量“有多大”的函数。注意它不只有一个定义，而是满足以下三条性质的一类函数，记作 $\|x\|$ ：

1. 正定性： $\|x\| \geq 0$ ，且 $\|x\| = 0$ 当且仅当 $x = 0$ （零向量的“大小”才是 0）
2. 齐次性： $\|\alpha x\| = |\alpha| \cdot \|x\|$ （向量放大几倍，大小也放大几倍）
3. 三角不等式： $\|x + y\| \leq \|x\| + \|y\|$ （两边之和大于等于第三边，和几何直觉一致）

L2 范数（欧几里得范数）——就是初中学的“距离”公式：

$$\|x\|_2 = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$$

```
In [152... u = torch.tensor([3.0, -4.0])
torch.linalg.norm(u)      # tensor(5.) ← 3²+4²=25, √25=5
```

Out[152... tensor(5.)

L1 范数 —— 所有元素绝对值之和:

$$\|x\|_1 = |x_1| + |x_2| + \dots + |x_n|$$

```
In [153... torch.abs(u).sum()      # tensor(7.) ← 3 + 4
```

Out[153... tensor(7.)

微积分

导数：瞬时变化率

$$f'(x) = \lim_{h \rightarrow 0} [f(x+h) - f(x)] / h$$

```
In [154... x = torch.tensor(2.0, requires_grad=True) # 要求 PyTorch 追踪 x 的梯度
y = x**3                                         # f(x) = x³
y.backward()                                    # 自动求导
print(x.grad)                                   # tensor(12.) ← 手算: 3x² = 3·2² = 12
```

tensor(12.)

偏导数

$$\nabla f(x) = (\partial f / \partial x_1, \partial f / \partial x_2, \dots, \partial f / \partial x_n)$$

```
In [155... x = torch.tensor([1.0, -1.0], requires_grad=True)
f = (x**2).sum()      # f = x₁² + x₂²
f.backward()
print(x.grad)         # tensor([2., -2.]) ← 手算 (2x₁, 2x₂) = (2, -2) ✓
```

tensor([2., -2.])

链式法则：复合函数求导

$$y = f(u), u = g(x) \rightarrow dy/dx = (dy/du) \cdot (du/dx)$$

例子: $y = (2x+1)^3$ 。设 $u = 2x+1$, 则 $dy/du = 3u^2$, $du/dx = 2$, 所以:

$$dy/dx = 3u^2 \cdot 2 = 6(2x+1)^2$$

自动微分

根据设计好的模型,系统会构建一个计算图,来跟踪计算那些数据通过那些操作组合起来产生输出.自动微分使系统能够随后反向传播梯度.这里反向传播意味着跟踪整个计算图,填充关于每个参数的偏导数

一个简单的例子

```
In [156... x=torch.arange(4.0)
x
```

```
Out[156... tensor([0., 1., 2., 3.])
```

在计算y关于x的梯度之前,需要一个地方来存储梯度.但是我们不会在每次对一个参数求导时都分配新的内存,因为我们经常会成千上万次更新相同的参数,每次都分配新的内存可能很快就会将内存耗尽

```
In [157... x.requires_grad_(True) # 等价于x=torch.arange(4.0,requires_grad=True)
x.grad # 默认值是None
```

```
In [158... y = 2 * torch.dot(x, x)
y
```

```
Out[158... tensor(28., grad_fn=<MulBackward0>)
```

```
In [159... y.backward()
x.grad
```

```
Out[159... tensor([ 0.,  4.,  8., 12.])
```

```
In [160... x.grad == 4 * x
```

```
Out[160... tensor([True, True, True, True])
```

```
In [161... # 在默认情况下, PyTorch会累积梯度, 我们需要清除之前的值
x.grad.zero_()
y = x.sum()
y.backward()
x.grad
```

```
Out[161... tensor([1., 1., 1., 1.])
```

概率

简单地说, **机器学习就是做出预测。**

- 根据病人的临床病史，预测他下一年心脏病发作的*概率*
- 异常检测中，评估一组发动机读数属于正常运行的概率
- 推荐系统中，估计用户购买某本书的概率

概率是一种灵活的语言，用于说明我们的**确定程度**。

以猫狗分类为例：

- 完全肯定图像是猫： $P(y=\text{"猫"}) = 1$
- 毫无证据时： $P(y=\text{"猫"}) = P(y=\text{"狗"}) = 0.5$
- 不十分确定： $0.5 < P(y=\text{"猫"}) < 1$

基本概率论

假设我们掷骰子，想知道看到 1 的几率。如果骰子是公平的，六个结果 $\{1, \dots, 6\}$ 等可能发生，因此 1 出现的概率为 $1/6$ 。

但现实中的骰子可能有瑕疵，检查的唯一方法是**多次投掷并记录结果**：

- 把某个值出现的次数 $\div$ 总投掷次数，作为该事件概率的**估计值**
- **大数定律**：随着投掷次数增加，估计值会越来越接近真实的潜在概率

```
In [162]: %matplotlib inline
import random
import torch
from torch.distributions import multinomial
import matplotlib.pyplot as plt
```

```
-----
ModuleNotFoundError                                Traceback (most recent call last)
Cell In[162], line 1
----> 1 get_ipython().run_line_magic('matplotlib', 'inline')
      2 import random
      3 import torch
      4 from torch.distributions import multinomial

ModuleNotFoundError: No module named 'matplotlib'
```

在统计学中，把从概率分布中抽取样本的过程称为**抽样**（sampling）。

- **分布**（distribution）：对事件的概率分配
- **多项分布**（multinomial distribution）：把概率分配给一些离散选择的分布

掷一次骰子 = 抽一个样本，输出向量的索引 i 处的值是采样结果中 $i+1$ 出现的次数：

```
In [ ]: fair_probs = torch.ones([6]) / 6 # 公平骰子：每个面概率 1/6
         multinomial.Multinomial(1, fair_probs).sample() # 掷一次
```

估计骰子公平性时，需要从同一分布生成**多个样本**。用框架的函数一次抽取多个样本（相当于掷 10 次骰子），比 Python 的 for 循环快得多：

```
In [ ]: multinomial.Multinomial(10, fair_probs).sample() # 掷 10 次，统计每个面出现的次数
```

模拟 1000 次投掷，统计每个数字被投中的次数，用**相对频率**作为真实概率的估计：

```
In [ ]: counts = multinomial.Multinomial(1000, fair_probs).sample()
counts / 1000 # 相对频率，每个值都应接近 1/6 ≈ 0.167
```

观察估计概率如何随数据增多而**收敛**到真实概率：进行 500 组实验，每组抽取 10 个样本（投 10 次）。

```
In [ ]: counts = multinomial.Multinomial(10, fair_probs).sample((500,))
cum_counts = counts.cumsum(dim=0) # 沿实验组累加
estimates = cum_counts / cum_counts.sum(dim=1, keepdims=True) # 累计相对频率

plt.figure(figsize=(6, 4.5))
for i in range(6):
    plt.plot(estimates[:, i].numpy(), label=("P(die=" + str(i + 1) + ")"))
plt.axhline(y=0.167, color='black', linestyle='dashed') # 真实概率 1/6
plt.xlabel('Groups of experiments')
plt.ylabel('Estimated probability')
plt.legend();
```

每条实线对应骰子的一个面，给出该面在每组实验后出现的估计概率。随着实验组数（数据量）增加，6 条曲线都向真实概率 $1/6 \approx 0.167$ 收敛——这就是**大数定律**。

概率论公理

- **样本空间** (sample space)：所有可能结果的集合，如掷骰子的 $S = \{1, 2, 3, 4, 5, 6\}$ ，其中每个元素是一个**结果** (outcome)
- **事件** (event)：给定样本空间中的一组随机结果，如“看到奇数” = $\{1, 3, 5\}$ 。若投出 3 点，因为 $3 \in \{1, 3, 5\}$ ，就称“看到奇数”这一事件发生了

概率是把集合映射到真实值的函数 $P(A)$ ，满足以下公理（科尔莫戈罗夫，1933）：

1. 对任意事件 A ，概率非负： $P(A) \geq 0$
2. 整个样本空间的概率为 1： $P(S) = 1$
3. 对**互斥**事件 $A_1, A_2, \dots$ （任意 $i \neq j$ 都有 $A_i \cap A_j = \emptyset$ ）具有可加性： $P(A_1 \cup A_2 \cup \dots) = P(A_1) + P(A_2) + \dots$

有了这个公理系统，可以避关于随机性的哲学争论，用数学语言严格推理。

随机变量

随机变量 (random variable) 几乎可以是任何数量，它在随机实验的一组可能性中取一个值。

- $P(X = 5)$ ：随机变量 X 取值为 5 的概率

- $P(X)$: 随机变量 X 上的**分布**，即 X 取各个值的概率
- $P(1 \leq X \leq 3)$: X 从 $\{1,2,3\}$ 中取值的概率

离散 vs **连续**随机变量：

- 离散：取值可枚举（如骰子的每一面）
- 连续：如身高、体重。测量两人身高是否**完全相同**意义不大；问"身高恰好是 1.80 米"的概率为 0，但**密度**不为 0；有意义的是落在区间（如 1.79~1.81 米）的概率

处理多个随机变量

很多场景需要同时考虑多个随机变量：疾病与症状的关系、图像的数百万个像素 + 标签 + 元数据（位置、时间、光圈等），它们都是**联合发生**的随机变量。

联合概率

联合概率 $P(A=a, B=b)$: $A=a$ 和 $B=b$ **同时**满足的概率。

$P(A=a, B=b) \leq P(A=a)$: 要同时发生， $A=a$ 必须发生，所以联合概率不会大于单个事件的概率。

条件概率

由联合概率引出的比率——**条件概率** $P(B=b | A=a)$: 在 $A=a$ 已发生的前提下， $B=b$ 的概率。

$$P(B=b | A=a) = P(A=a, B=b) / P(A=a)$$

贝叶斯定理

由乘法法则 $P(A,B) = P(B|A) \cdot P(A)$ 与对称性 $P(A,B) = P(A|B) \cdot P(B)$ ，可得统计学中最有用的公式之一：

$$P(A|B) = P(B|A) \cdot P(A) / P(B)$$

其中 $P(A,B)$ 是**联合分布**， $P(A|B)$ 是**条件分布**。它让我们能在"原因导致结果"与"结果反推原因"之间转换概率。

边际化

求和法则：B 的概率等于对 A 的所有可能取值，把联合概率聚合起来：

$$P(B) = \sum_A P(A, B)$$

这也称为**边际化**（marginalization），结果称为**边际概率** / **边际分布**。

独立性

如果 A 的发生与 B 无关，称 A 与 B **独立**（记作 $A \perp B$ ），此时：

$$P(A|B) = P(A) \iff P(A, B) = P(A) \cdot P(B)$$

（两个随机变量独立，当且仅当联合分布等于各自分布的乘积）

- 独立：连续两次掷骰子
- 依赖：灯开关的位置与房间亮度（可能灯泡坏了、电源故障）

条件独立 $A \perp B \mid C$ ：给定 C 时， $P(A, B|C) = P(A|C) \cdot P(B|C)$

应用：HIV 检测（贝叶斯定理实战）

假设一项 HIV 检测相当准确：

- 患者健康（ $H=0$ ）但测试**误报**为阳性的概率只有 1%： $P(D_1=1 \mid H=0) = 0.01$
- 患者真感染（ $H=1$ ）时，测试**绝不会漏报**： $P(D_1=1 \mid H=1) = 1$
- 疾病本身罕见，先验概率 $P(H=1) = 0.0015$

问题：一个人检测出阳性，他真的感染 HIV 的概率有多大？

```
In [ ]: P_H = torch.tensor([1 - 0.0015, 0.0015]) # 先验: P(H=0 健康), P(H=1 感染)
P_D1_given_H = torch.tensor([0.01, 1.0]) # 似然: P(D1=1|H=0)=0.01 误报, P(D1=1|H=1)=1

# 分母用边际化求 P(D1=1)，再代入贝叶斯定理
P_D1 = torch.dot(P_D1_given_H, P_H)
P_H_given_D1 = P_D1_given_H * P_H / P_D1
P_H_given_D1 # tensor([0.8693, 0.1306])
```

尽管检测非常准确，阳性结果下真正感染的概率只有约 **13.06%**！

原因：疾病极其罕见，健康人基数太大，1% 的误报产生的**假阳性**人数远多于真阳性。

改进：再做一次独立检测 D_2 。 两次独立检测都误报的概率只有 $0.01 \times 0.01 = 0.0001$ ：

```
In [ ]: P_D2_and_D1_given_H = torch.tensor([0.0001, 1.0]) # 两次独立误报: 0.01 * 0.01
P_D1_and_D2 = torch.dot(P_D2_and_D1_given_H, P_H)
P_H_given_D1_and_D2 = P_D2_and_D1_given_H * P_H / P_D1_and_D2
P_H_given_D1_and_D2 # tensor([0.0624, 0.9376])
```

第二次检测把置信度从约 13.06% 提升到约 **93.76%**（错误率降到约 6.24%）。

核心结论：**条件概率的正确解读依赖先验**。即使检测手段很准，对极罕见的事件，单次检测的阳性结果也不足以下定论。

期望和方差

期望 (expectation) : 随机变量按概率加权的平均值, 刻画分布的中心位置。

$$E[X] = \sum x \cdot P(X = x)$$

方差 (variance) : 随机变量偏离期望的程度 (偏差平方的期望) 。

$$\text{Var}(X) = E[(X - \mu)^2] = E[X^2] - \mu^2, \text{ 其中 } \mu = E[X]$$

- 方差的平方根称为**标准差** (统计学家) 或**均方根 RMS** (工程师)
- 方差越小, 随机变量的取值越集中在期望附近

```
In [ ]: # 以公平骰子为例:  $E[X] = 3.5$ ,  $\text{Var}(X) = 35/12 \approx 2.9167$ 
x = torch.arange(1., 7.)          # 骰子的 6 个取值
p = torch.ones(6) / 6             # 每个取值的概率

mu = (p * x).sum()                 # 期望  $E[X]$ 
var = (p * x**2).sum() - mu**2     # 方差 =  $E[X^2] - \mu^2$ 
mu, var
```